STRUCTURAL ASSESSMENT

orchard

Every number in this document was measured from the project files, the source text and the change history. Nothing was estimated, and no part of it was written by a language model. What could not be measured is listed at the end rather than left out.

IN SHORT

orchard is 89 projects and 414,611 lines across 6,203 source files. Everything targets v4.8. No test project references 45 of the 78 projects that ship code, covering 75,908 lines. 449 files were ranked on structure and change frequency together. The one at the top is src/Orchard.Web/Modules/Orchard.OutputCache/Filters/OutputCacheFilter.cs. 89 of 89 project files are in the pre-SDK format. 73 projects reference packages that exist only inside the .NET Framework, which no conversion tool will fix; 16 others are convertible as they stand. The work is ordered from what blocks everything else through to what may cost nothing, and starts with: classify the packages nobody has checked yet.

414,611
lines of code
in 6,203 source files

89
projects
all targeting v4.8

75,908
lines in projects no test project references
45 of the 78 projects that ship code

7
packages that exist only inside the .NET Framework
73 projects blocked by them

16
projects convertible as they stand
89 of 89 in the old project format

WHERE THE ORDER OF WORK STARTS

1. **Classify the packages nobody has checked yet**
   Needed before anyone can price the rest. 78 packages.
2. **Convert the project files that can take the modern format**
   Mechanical, no decision required. 13 projects.
3. **Retire the hand-written binding redirects**
   Mechanical, no decision required. 279 redirects.

The whole order, with the reasons and what each step covers, starts on page 11.

orchard

A structural assessment, generated on 2026-10-19 by Legacy Lens.

Every number in this document was measured from the project files, the source text and the change history. Nothing was estimated, and no part of it was written by a language model. What could not be measured is listed at the end rather than left out.

In short

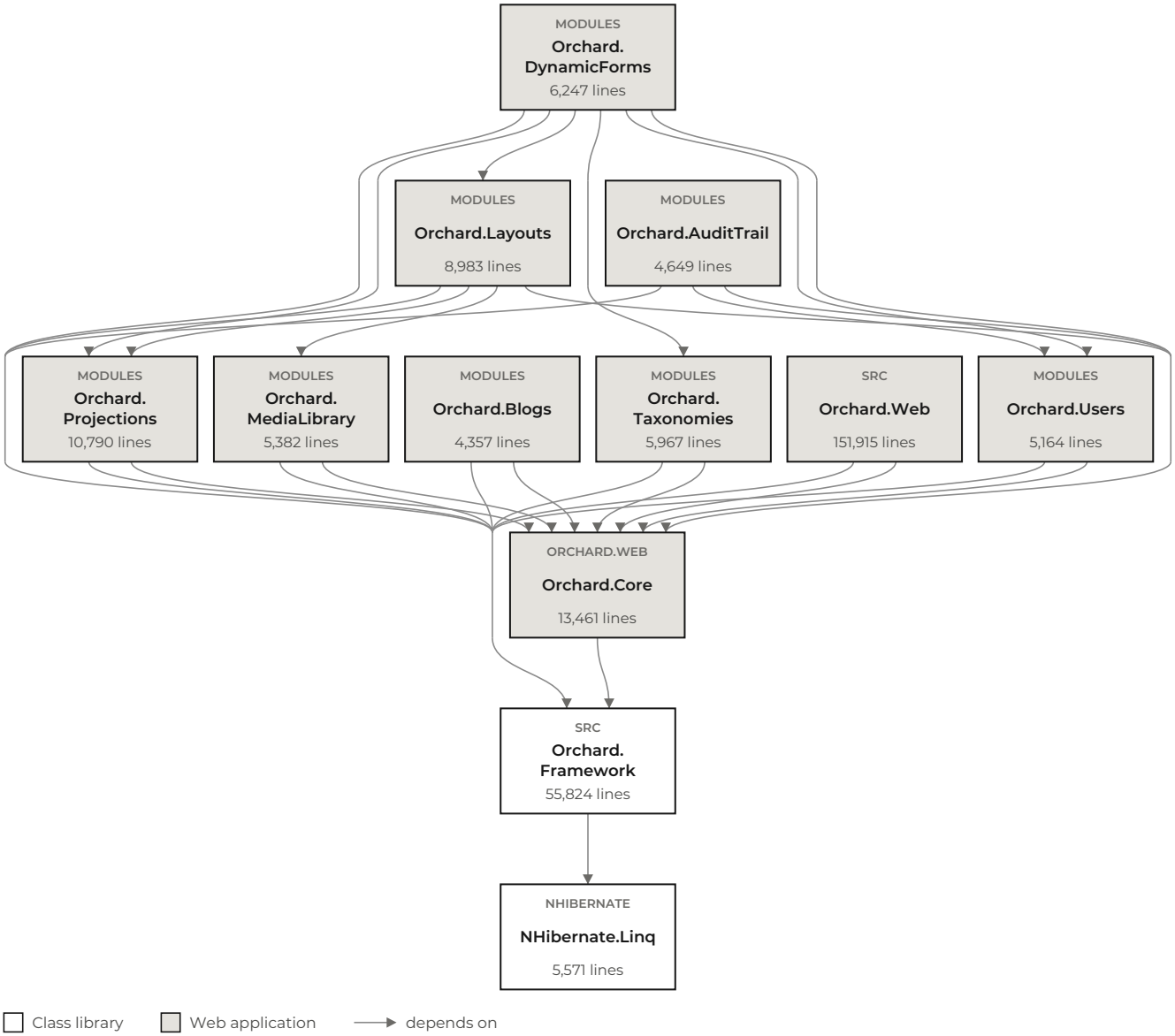
orchard is 89 projects and 414,611 lines across 6,203 source files. Everything targets v4.8. No test project references 45 of the 78 projects that ship code, covering 75,908 lines. 449 files were ranked on structure and change frequency together. The one at the top is src/Orchard.Web/Modules/Orchard.OutputCache/Filters/OutputCacheFilter.cs. 89 of 89 project files are in the pre-SDK format. 73 projects reference packages that exist only inside the .NET Framework, which no conversion tool will fix; 16 others are convertible as they stand. The work is ordered from what blocks everything else through to what may cost nothing, and starts with: classify the packages nobody has checked yet.

What this is

Kind	Projects	Lines	Files
Web application	73	303,718	5,014
Class library	4	62,255	854
Test project	11	47,570	317
Console application	1	1,068	18
Total	**89**	**414,611**	**6,203**

The projects declare 319 dependencies on each other. Project kind is decided by what sits in the folder rather than by which assemblies are referenced, because references lie and a web.config next to a Views folder does not.

The 12 projects over 4,357 lines, and how they depend on each other. 77 projects are left out: the test projects, and everything smaller than that. A diagram of all 89 renders as a wall and tells a reader nothing.



Drawn from the Mermaid source the Markdown report carries, with what each project depends on below it.

The largest projects, which is where the time goes:

Project	Kind	Target	Lines
`Orchard.Web`	Web application	v4.8	151,915
`Orchard.Framework`	Class library	v4.8	55,824
`Orchard.Framework.Tests`	Test project	v4.8	23,163
`Orchard.Core`	Web application	v4.8	13,461
`Orchard.Projections`	Web application	v4.8	10,790
`Orchard.Specs`	Test project	v4.8	10,069
`Orchard.Tests.Modules`	Test project	v4.8	9,916
`Orchard.Layouts`	Web application	v4.8	8,983
`Orchard.DynamicForms`	Web application	v4.8	6,247
`Orchard.Taxonomies`	Web application	v4.8	5,967

What will hurt

Finding	Count	Example
Untested project	45	`NHibernate.Linq`: No test project references it (5,571 lines)
Oversized project	3	`Orchard.Framework.Tests`: 23,163 lines in one project
Library coupled to the web stack	1	`Orchard.Framework`: Library depends on System.Web.Http, System.Web.Http.WebHost, System.Web.Mvc, System.Web.WebPages, System.Web.WebPages.Deployment, System.Web.WebPages.Razor
Referenced by nothing	1	`MSBuild.Orchard.Tasks`: Nothing in the solution references it

Untested project. Changes here are verified by running the application, or not at all.

Oversized project. Past roughly 20,000 lines nobody holds the whole thing in their head, and every change is made without seeing the consequences.

Library coupled to the web stack. It cannot be unit tested without a web context, and cannot be reused from a service, a desktop client or a batch job. Usually found out the day someone tries.

Referenced by nothing. Either dead code, or loaded at runtime by reflection or a plugin mechanism, which the project files cannot show.

The files most likely to break

Ranked on three signals that were all already on disk: complexity from the syntax tree, change frequency from git over the full history, and whether a test appears to cover the file. The score is the geometric mean of structure and churn, so a file has to score high on both to reach the top: complicated but never touched is not urgent, and touched constantly but trivial is not dangerous. The score is a sort key, and the reasons beside it are the argument. Generated code and everything inside a test project are left out rather than ranked, because reporting that a test fixture is complicated and untested is true and useless.

The history window was widened. Only 102 of this directory's 11677 commits fall in the last 24 months, so the whole history was read instead. A window that recent describes a codebase that has stopped changing by its tail.

Score	File	Lines	Commits	Tested	Why
1.23	`src/Orchard.Web/Modules/Orchard.OutputCache/Filters/OutputCacheFilter.cs`	705	82	**no**	changed in 82 commits; no test file appears to cover it
1.23	`src/Orchard.Web/Core/Shapes/CoreShapes.cs`	866	79	**no**	Pager_Links has a cyclomatic complexity of 34: covering its branches would take 34 tests; changed in 79 commits; no test file appears to cover it
1.01	`src/Orchard/ContentManagement/DefaultContentQuery.cs`	386	44	**no**	nested 6 levels deep; changed in 44 commits; no test file appears to cover it
0.98	`src/Orchard.Web/Modules/Orchard.Taxonomies/Services/TaxonomyService.cs`	564	34	**no**	changed in 34 commits; no test file appears to cover it
0.97	`src/Orchard/ContentManagement/DefaultHqlQuery.cs`	724	28	**no**	ToHql has a cyclomatic complexity of 23: covering its branches would take 23 tests; changed in 28 commits; no test file appears to cover it
0.97	`src/Orchard/Data/Migration/Interpreters/DefaultDataMigrationInterpreter.cs`	407	43	**no**	changed in 43 commits; no test file appears to cover it
0.95	`src/Orchard/ContentManagement/DefaultContentManager.cs`	780	127	yes	Get has a cyclomatic complexity of 25: covering its branches would take 25 tests; nested 7 levels deep; changed in 127 commits

Score	File	Lines	Commits	Tested	Why
0.91	`src/Orchard.Web/Modules/Upgrade/Controllers/InfosetController.cs`	589	33	**no**	changed in 33 commits; no test file appears to cover it
0.89	`src/Orchard.Web/Core/Contents/Controllers/AdminController.cs`	499	119	yes	changed in 119 commits
0.87	`src/Orchard/ContentManagement/Drivers/ContentPartDriver.cs`	239	47	**no**	changed in 47 commits; no test file appears to cover it
0.87	`src/Orchard.Web/Modules/Orchard.Blogs/Controllers/BlogPostAdminController.cs`	252	48	**no**	changed in 48 commits; no test file appears to cover it
0.86	`src/Orchard/Environment/ShellBuilders/ShellContainerFactory.cs`	301	35	**no**	CreateContainer has a cyclomatic complexity of 35: covering its branches would take 35 tests; nested 6 levels deep; changed in 35 commits; no test file appears to cover it
0.86	`src/Orchard.Web/Modules/Orchard.Users/Controllers/AccountController.cs`	612	88	yes	changed in 88 commits
0.85	`src/Orchard.Web/Modules/Orchard.ContentTypes/Services/ContentDefinitionService.cs`	354	46	**no**	changed in 46 commits; no test file appears to cover it
0.85	`src/Orchard.Web/Modules/Orchard.Azure/Services/FileSystems/AzureFileSystem.cs`	419	28	**no**	changed in 28 commits; no test file appears to cover it

What a move to modern .NET runs into

	Count
Projects	89
In the old project format	89
Convertible as they stand	16
Blocked by a package with no path forward	73
Package references in total	722
Distinct packages	93
Pinned to more than one version	0
Hand-written binding redirects	279

This legacy has drifted. Every package is pinned to a single version today, but 279 binding redirects were written by hand. A redirect only ever gets written because two assemblies disagreed, so they are the trace of drift that has already been paid for once. Old and drifted is a different job from old but coherent, and the difference belongs in the price rather than in the reader's head.

Packages that exist only inside the .NET Framework:

Package	Projects
`Microsoft.AspNet.Mvc`	73
`Microsoft.AspNet.Razor`	73
`Microsoft.AspNet.WebPages`	73
`Microsoft.Web.Infrastructure`	73
`Microsoft.CodeDom.Providers.DotNetCompilerPlatform`	61
`Microsoft.Owin.Host.SystemWeb`	5
`Microsoft.AspNet.WebApi.WebHost`	1

No version of these runs on modern .NET, so no conversion tool will help. The code that calls them is rewritten, replaced, or left where it is.

78 packages are reported as unclassified rather than assumed to be fine. Naming a package portable without checking is how a quote acquires the one item that doubles it.

What you actually use of what is going away

A package with no future is not a unit of work. What decides the size is how much of it this codebase touches and how few types carry it: a hundred calls through six types is an afternoon, and the same hundred spread over sixty is a rewrite. Counted from the syntax, so a type is counted where it appears in a type position in a file importing the package and nothing in the solution declares it.

Microsoft.AspNet.Mvc

3,619 uses of 185 types, across 365 files. 24 types and 134 files carry four fifths of it.

- `ActionResult` (541 uses)
- `HttpUnauthorizedResult` (408 uses)
- `HttpPost` (260 uses)
- `RouteValueDictionary` (194 uses)
- `SelectListItem` (172 uses)
- `HtmlHelper` (167 uses)
- `ActionName` (134 uses)
- `Controller` (131 uses)

Microsoft.AspNetCore.Mvc covers 83% of those calls. 116 types, 543 calls, are not in the catalogue at all: unknown, which is not the same as fine.

1 type, 57 calls, are deleted rather than replaced: the framework withdrew the feature, so the use goes and nothing takes its place. They are named, because a deletion somebody has not agreed to is still a change to their code:

- `ValidateInput` (57 uses)

Asked of the framework itself rather than of the catalogue: 7 types of that column exist inside Microsoft.AspNetCore.Mvc under the same name, which is a lead worth checking. 10 types exist elsewhere in the framework under the same name, which nobody has recorded either way. 99 types the framework does not have at all. That leaves 109 types still to decide.

Those are worth a minute each, because the framework was split up and a counterpart in a sibling package looks the same from here as a coincidence:

- `HttpContext` (8 uses) is `Microsoft.AspNetCore.Http.HttpContext`
- `IRouteHandler` (5 uses) is `Microsoft.AspNetCore.Routing.IRouteHandler`
- `VirtualPathData` (5 uses) is `Microsoft.AspNetCore.Routing.VirtualPathData`
- `HttpRequest` (3 uses) is `Microsoft.AspNetCore.Http.HttpRequest`
- `IRouteConstraint` (2 uses) is `Microsoft.AspNetCore.Routing.IRouteConstraint`
- `RouteDirection` (2 uses) is `Microsoft.AspNetCore.Routing.RouteDirection`
- `AuthenticationProperties` (1 use) is `Microsoft.AspNetCore.Authentication.AuthenticationProperties`
- `HttpResponse` (1 use) is `Microsoft.AspNetCore.Http.HttpResponse`

3 types, 5 calls, have no replacement recorded at all. Those are the ones that decide whether this is a migration or a rewrite, and they are named rather than counted:

- `OutputCacheAttribute` (3 uses)
- `HandleErrorAttribute` (1 use)

- `IDependencyResolver` (1 use)

Autofac

790 uses of 99 types, across 159 files. 28 types and 67 files carry four fifths of it.

- `ContainerBuilder` (144 uses)
- `Mock` (127 uses)
- `ILifetimeScope` (60 uses)
- `IComponentRegistration` (34 uses)
- `HttpContextBase` (24 uses)
- `RequestContext` (22 uses)
- `HtmlString` (21 uses)
- `IComponentContext` (17 uses)

Nobody has recorded what replaces this. That is a gap in the catalogue rather than a statement about the package.

Newtonsoft.Json

510 uses of 23 types, across 31 files. 4 types and 2 files carry four fifths of it.

- `JProperty` (243 uses)
- `JObject` (114 uses)
- `JArray` (38 uses)
- `JToken` (22 uses)
- `ActionResult` (17 uses)
- `HttpUnauthorizedResult` (16 uses)
- `HtmlString` (12 uses)
- `HttpPost` (11 uses)

System.Text.Json covers 0% of those calls. 17 types, 91 calls, are not in the catalogue at all: unknown, which is not the same as fine.

Asked of the framework itself rather than of the catalogue: 0 types of that column exist inside System.Text.Json under the same name, which is a lead worth checking. 7 types exist elsewhere in the framework under the same name, which nobody has recorded either way. 10 types the framework does not have at all. That leaves 17 types still to decide.

Those are worth a minute each, because the framework was split up and a counterpart in a sibling package looks the same from here as a coincidence:

- `ActionResult` (17 uses) is `Microsoft.AspNetCore.Mvc.ActionResult`
- `HtmlString` (12 uses) is `Microsoft.AspNetCore.Html.HtmlString`
- `Controller` (3 uses) is `Microsoft.AspNetCore.Mvc.Controller`
- `FormCollection` (3 uses) is `Microsoft.AspNetCore.Http.FormCollection`
- `JsonResult` (1 use) is `Microsoft.AspNetCore.Mvc.JsonResult`
- `RouteData` (1 use) is `Microsoft.AspNetCore.Routing.RouteData`
- `SelectListItem` (1 use) is `Microsoft.AspNetCore.Mvc.Rendering.SelectListItem`

5 types, 418 calls, have no replacement recorded at all. Those are the ones that decide whether this is a migration or a rewrite, and they are named rather than counted:

- `JProperty` (243 uses)
- `JObject` (114 uses)

- `JArray` (38 uses)
- `JToken` (22 uses)
- `JValue` (1 use)

NHibernate

462 uses of 79 types, across 58 files. 25 types and 28 files carry four fifths of it.

- `ICriteria` (76 uses)
- `ISession` (50 uses)
- `ICriterion` (30 uses)
- `Mock` (29 uses)
- `ISessionFactory` (25 uses)
- `Configuration` (23 uses)
- `ContainerBuilder` (17 uses)
- `IProjection` (16 uses)

Nobody has recorded what replaces this. That is a gap in the catalogue rather than a statement about the package.

Microsoft.AspNet.WebApi.Core

92 uses of 38 types, across 13 files. 23 types and 7 files carry four fifths of it.

- `ContainerBuilder` (8 uses)
- `ILifetimeScope` (6 uses)
- `IHttpController` (5 uses)
- `RouteCollection` (5 uses)
- `ConfigurationSettingsReader` (4 uses)
- `HttpConfiguration` (4 uses)
- `HttpControllerContext` (4 uses)
- `HttpControllerDescriptor` (4 uses)

Microsoft.AspNetCore.Mvc covers 7% of those calls. 33 types, 82 calls, are not in the catalogue at all: unknown, which is not the same as fine.

Asked of the framework itself rather than of the catalogue: 5 types of that column exist inside Microsoft.AspNetCore.Mvc under the same name, which is a lead worth checking. 4 types exist elsewhere in the framework under the same name, which nobody has recorded either way. 24 types the framework does not have at all. That leaves 28 types still to decide.

Those are worth a minute each, because the framework was split up and a counterpart in a sibling package looks the same from here as a coincidence:

- `RouteCollection` (5 uses) is `Microsoft.AspNetCore.Routing.RouteCollection`
- `KeyedService` (2 uses) is `Microsoft.Extensions.DependencyInjection.KeyedService`
- `RouteBase` (1 use) is `Microsoft.AspNetCore.Routing.RouteBase`
- `RouteData` (1 use) is `Microsoft.AspNetCore.Routing.RouteData`

1 type, 4 calls, have no replacement recorded at all. Those are the ones that decide whether this is a migration or a rewrite, and they are named rather than counted:

- `HttpConfiguration` (4 uses)

4 further catalogued package(s) are used and not listed here. The command prints all of them.

What there is to decide about the frameworks themselves

The names nothing accounts for come to 4 questions. What follows is what each one was for, what replaced it, and what there is to choose between. None of the options is a recommendation: which one is right depends on how many people, how much time, and whether the old and the new have to run side by side, and none of that is in the code.

The request context. 109 calls through `RequestContext`, `HttpContextBase`, `HttpRequestBase`, `HttpResponseBase`, `HttpCookieCollection`, `HttpContextWrapper`, and 3 more.

System.Web wrapped the request in a family of abstract base classes so that code could be tested against them, and passed one around as the ambient state of the request. ASP.NET Core has HttpContext and its parts, injected rather than ambient, with different names and a different shape. An uploaded file is IFormFile, cookies are IRequestCookieCollection, and there is no HttpException.

- Move the code onto the new types directly. Shortest where the class already receives the context as an argument.
- Put an interface of your own in front of what you actually use, and implement it over the new types. Longest, and the only option that lets the old and the new code run side by side.
- Push the context out of the class entirely and pass the values it reads. Turns a migration into a design change, which is sometimes the right trade and never a small one.

Pipeline internals. 72 calls through `MvcRouteHandler`, `HttpApplication`, `MvcHandler`, `ControllerBuilder`, `ReflectedControllerDescriptor`.

Code that reached into how MVC found controllers, resolved views or handled the application lifetime. Global.asax is the usual entry point. There is no equivalent, and that is deliberate: the pipeline is middleware you compose in Program.cs rather than handlers you replace.

- Find what the code was working around and check whether the new pipeline does it already. Most of these predate features the framework grew.
- Write it as middleware, which is where the behaviour belongs now.
- Treat it as a rewrite of that concern, and price it as one. This is the group that genuinely costs.

Areas. 7 calls through `IRouteWithArea`.

An area registered itself by deriving from AreaRegistration, and the framework found it by scanning assemblies at startup. Areas still exist and there is nothing to register: a controller carries [Area] and routes name it.

- Delete the registration classes and put [Area] on the controllers.
- Ask first whether the areas earn their keep, because the routing they were working around is gone.

Model binding extensibility. 6 calls through `ModelBinderDictionary`, `DefaultModelBinder`.

Binding was extended by deriving from the default binder or replacing the metadata provider, both of which knew a great deal about the pipeline around them. ASP.NET Core binds through IModelBinder and IModelBinderProvider, registered in options. The shape is similar and the base classes are not there to derive from.

- Write the binder against the new interfaces. Usually smaller than the original, because the default no longer has to be reimplemented around it.
- Check first whether the framework now does it. Several of these existed to bind types that bind on their own today.

What only you can decide

Everything below came out of preparing the mechanical work, and none of it can be settled by reading the code harder. They are listed before the order because the order assumes them.

73 projects, one shown. Still depends on packages with no path to modern .NET: Microsoft.AspNet.Mvc, Microsoft.AspNet.Razor, Microsoft.AspNet.WebApi.WebHost, Microsoft.AspNet.WebPages, Microsoft.Web.Infrastructure. This changes the file format and not the future of the project.

Orchard.Framework, Orchard.Specs, Orchard.Tests.Modules, Orchard.Framework.Tests, Orchard.Core.Tests, Orchard.ContentTypes, and 67 more.

Across the solution. 1 key(s) are declared in more than one config file with different values, and the last one read wins here: aspnet:RoslynCompilerLocation. Which one was right at runtime depended on which application loaded them, and that is not something these files record.

Across the solution. 1 key(s) are read by the code and declared nowhere. They are not in this file either, because inventing a value is the one thing that would make it wrong.

In what order

This is a dependency order, not a schedule, and it deliberately carries no days: nothing here measured how fast anyone works. What it does carry is a property of the codebase. Work that blocks the rest comes first, because the report itself is unreliable while it stands. Mechanical work comes before decisions, because it is the half nobody has to argue about and doing it shrinks the surface the arguments are about.

1. Classify the packages nobody has checked yet

Needed before anyone can price the rest. 78 packages.

These are neither known to work on modern .NET nor known to be dead ends. Each one is either a non-event or a rewrite, and which of the two decides the size of everything below. Quoting before they are classified means quoting on the assumption that none of them is a problem.

- `Antlr3.Runtime (22 projects)`
- `Iesi.Collections (22 projects)`
- `Remotion.Linq.EagerFetching (22 projects)`
- `System.Buffers (9 projects)`
- `System.Memory (9 projects)`
- `System.Numerics.Vectors (9 projects)`
- `System.Runtime.CompilerServices.Unsafe (9 projects)`
- `System.Threading.Tasks.Extensions (8 projects)`
- `FluentNHibernate (7 projects)`
- `Microsoft.Owin (7 projects)`

2. Convert the project files that can take the modern format

Mechanical, no decision required. 13 projects.

Only the file format is old, and the conversion is a tool run and a review. It is worth doing first because it shrinks what everything after this has to work through. Some of these still depend on packages with no path forward: that is a separate problem and the format change does not pretend to fix it. What comes out is a patch that applies, and whether the solution restores and compiles afterwards is for the build to say, not for this.

- `MSBuild.Orchard.Tasks`
- `NHibernate.Linq`
- `Orchard`
- `Orchard.Azure.Tests`
- `Orchard.Framework`
- `Orchard.Messaging.Tests`
- `Orchard.Profile`
- `Orchard.Projections.Tests`
- `Orchard.Specs`
- `Orchard.Tests`

3. Retire the hand-written binding redirects

Mechanical, no decision required. 279 redirects.

A redirect only ever gets written because two assemblies disagreed about a version. The project files no longer show the disagreement, which means the redirects are load-bearing: removing one at conversion time is how a migration acquires a runtime failure that nothing in the build predicted.

4. Decide what happens to the code built on System.Web

Needs a decision, not a tool. 73 projects.

7 packages here exist only inside the .NET Framework. No version of them runs on modern .NET, so no conversion tool will help: the code that calls them is rewritten, replaced, or left where it is. That is a decision about the product, not about the code, and it is the one that decides whether the rest is a migration or a rewrite.

- `Microsoft.AspNet.Mvc (73 projects)`
- `Microsoft.AspNet.Razor (73 projects)`
- `Microsoft.AspNet.WebPages (73 projects)`
- `Microsoft.Web.Infrastructure (73 projects)`
- `Microsoft.CodeDom.Providers.DotNetCompilerPlatform (61 projects)`
- `Microsoft.Owin.Host.SystemWeb (5 projects)`
- `Microsoft.AspNet.WebApi.WebHost (1 project)`

5. Separate the libraries that depend on the web stack

Needs a decision, not a tool. 1 library.

A library holding a web dependency cannot be tested without a web context, and cannot be reused from a service, a desktop client or a batch job. It is listed after the decision above because what the web layer becomes decides how much of this is worth untangling.

- `Orchard.Framework: Library depends on System.Web.Http, System.Web.Http.WebHost, System.Web.Mvc, System.Web.WebPages, System.Web.WebPages.Deployment, System.Web.WebPages.Razor`

6. Put tests around the files most likely to break

Never finished, only kept up. 20 files.

No test project references 45 projects in this solution. The files below are the ones where that matters most: they rank highest on complexity and change frequency together, which is the combination that produces incidents. This is the one item here that is never finished, which is why it starts early and runs alongside the rest.

- `src/Orchard.Web/Modules/Orchard.OutputCache/Filters/OutputCacheFilter.cs (score 1.23)`
- `src/Orchard.Web/Core/Shapes/CoreShapes.cs (score 1.23)`
- `src/Orchard/ContentManagement/DefaultContentQuery.cs (score 1.01)`
- `src/Orchard.Web/Modules/Orchard.Taxonomies/Services/TaxonomyService.cs (score 0.98)`
- `src/Orchard/ContentManagement/DefaultHqlQuery.cs (score 0.97)`
- `src/Orchard/Data/Migration/Interpreters/DefaultDataMigrationInterpreter.cs (score 0.97)`
- `src/Orchard.Web/Modules/Upgrade/Controllers/InfosetController.cs (score 0.91)`
- `src/Orchard/ContentManagement/Drivers/ContentPartDriver.cs (score 0.87)`
- `src/Orchard.Web/Modules/Orchard.Blogs/Controllers/BlogPostAdminController.cs (score 0.87)`
- `src/Orchard/Environment/ShellBuilders/ShellContainerFactory.cs (score 0.86)`

7. Confirm whether the unreferenced projects are dead

May cost nothing. 1 project.

Nothing in the solution references these. They are either dead code, which makes them the cheapest work in this report, or they are loaded at runtime by reflection or a plugin mechanism, which project files cannot show. Someone who knows the product answers this in minutes.

- `MSBuild.Orchard.Tasks`

What this report does not say

Printed here rather than left for the reader to discover after acting on it. Each of these applies to this codebase; the ones that did not apply were left out.

Nothing was compiled. The whole report is read from project files, source text and git. That is what lets it run on a solution that does not build, which is the state most inherited code is in. The cost is that anything resolved at runtime, by reflection, dependency injection or a plugin loader, is invisible here.

Test coverage is a naming convention. A file is counted as tested when a test file appears to be named after it. That over-reports coverage and never under-reports it: a file called untested here is genuinely untested, while one called tested may only share a name with a test that exercises none of it.

Most packages are unclassified. 78 of 93 distinct packages are neither known to run on modern .NET nor known to be dead ends. They are reported as unknown rather than assumed to be fine, because a survey that quietly counts an unknown package as portable is one that discovers the problem after the price has been agreed.

Generated code was excluded. 31 generated files were left out of the ranking. They top every chart on size and complexity and tell a reader nothing they can act on, but they are still code that ships, and a regeneration path that no longer exists is its own problem.

Measured in 7,673 ms, without compiling anything, without restoring a package and without contacting any service. That is what makes it usable on a solution that does not build, which is the state an inherited codebase is in on the first day. Generated by Legacy Lens.